Q3

- الناتج = عدد الباحثين × عدد الأبحاث (Cartesian product)، ممكن ملايين الصفوف.
- استهلاك كبير في الـ CPU والـ memory والـ network.
- النتيجة مالهاش معنى لأن مفيش علاقة بين الصفوف (الربط الصح عن طريق Authors).

Q8

- بيرجعوا نفس أول 3 صفوف، لكن مش متطابقين في الإمكانيات.
- T-SQL: TOP فقط، بيدعم WITH TIES و PERCENT، مفيهوش Skip.
- ANSI standard: OFFSET-FETCH، لازم ORDER BY، بيدعم Paging (تخطي صفوف)، وبيقبل variables و expressions.

Q11

- RANK: المتعادلين ليهم نفس الترتيب وبعدهم فجوة (1,1,3).
- DENSE_RANK: نفس الترتيب من غير فجوة (1,1,2).

Q13

FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY → TOP/OFFSET-FETCH

Q14

- WHERE بتتنفذ قبل GROUP BY و SELECT، فمعندهاش الـ alias ومينفعش فيها aggregate.
- الحل: HAVING COUNT(*) > 2 (موجود في الكود).

Q33

Multi-statement TVF أحسن لما:

- المنطق محتاج أكتر من statement (variables، IF، WHILE، حسابات وسيطة).
- محتاجين نعرّف شكل الجدول المرتجع وننضف البيانات قبل ما نرجعها.
- الـ Inline أسرع لأن الـ optimizer بيتعامل معاه زي الـ view، أما الـ Multi-statement فبيتعامل كـ black box.

Q35

- Clustered: بيرتب الصفوف الفعلية في الجدول، والـ leaf هو الداتا نفسها.
- Non-clustered: هيكل منفصل فيه الـ key + مؤشر للصف الأصلي.
- الجدول فيه Clustered واحد بس.
- لما نعمل Clustered، كل الـ Non-clustered الموجودة بتتعمل لها rebuild لأن الـ row locator بيتغير من RID لـ clustered key.

Q36

- Covering Index: فيه كل الأعمدة اللي الاستعلام محتاجها (key + INCLUDE).
- بيتجاوب من الـ index لوحده من غير Key Lookup، فـ I/O أقل بكتير.

Q39

- المتطلبات: SCHEMABINDING، أسماء بجزئين (dbo.Table)، أول index لازم UNIQUE CLUSTERED، COUNT_BIG(*) مع GROUP BY، نفس الـ database والـ SET options، owner معينة.
- القيود: ممنوع AVG/MIN/MAX، ORDER BY، DISTINCT، TOP، UNION، subqueries، OUTER JOIN، والدوال non-deterministic، ومعاه HAVING.
- الفوائد: النتيجة متخزنة فالقراءة سريعة، والـ optimizer يستخدمه حتى لو الاستعلام على الجداول الأصلية.
- نختاره بدل الـ view العادي لما الاستعلام تقيل ومتكرر والقراءة أكتر من الكتابة، وبدل الجدول لأنه بيتحدث تلقائياً.

- العيب: بيبطّأ INSERT/UPDATE/DELETE ويحتاج مساحة.

Q44

- اخترت AFTER: التعديل الأصلي لـ EndDate يتنفذ الأول وبعدها الـ trigger يعدل Status.
- INSTEAD OF كان هيلغي التعديل الأصلي ونضطر نعيد كتابته بنفسنا.

Q47

- Atomicity: العملية كلها أو ولا حاجة. مثال: لو تخصيص المنحة فشل، المشروع والتعيين يتلغوا.
- Consistency: الداتا تفضل صحيحة. مثال: مينفعش نضيف WorksOn لباحث أو مشروع مش موجود.
- Isolation: العمليات المتزامنة متأثرش على بعض. مثال: موظفين بيعدلوا TotalAmount لنفس الـ Grant ومفيش تعديل بيضيع.
- Durability: بعد COMMIT الداتا بتفضل حتى لو السيرفر وقع. مثال: عدد CitationCount بعد تأكيده.

Q48

- غالباً نفس الأداء، الـ optimizer بيحوّل الاتنين لـ Semi Join.
- EXISTS بيقف عند أول تطابق وأحسن مع NULLs.
- العوامل: الـ indexes، الـ statistics، حجم الجداول، عدد صفوف الـ subquery، وجود NULLs.

Q49

- Index على ResearchProject(Status, StartDate DESC) بيغطي الفلترة والترتيب ويمنع Sort.
- Index على WorksOn(ProjectID) عشان الـ join.
- INNER JOIN بدل LEFT JOIN لو مش محتاجين المشاريع اللي ملهاش باحثين.
- UPDATE STATISTICS.

Q50

- A - كتابة الاستعلام كل مرة: تكرار كود وصعب الصيانة، الداتا fresh.
- B - View: مركزي وسهل الصيانة وفيه security، الداتا fresh، الأداء زي الاستعلام العادي.
- C - Stored Procedure: plan متخزن وبيقبل parameters، لكن مينفعش نعمل عليه SELECT مباشرة.
- D - Indexed View: مش ممكن هنا لأن TOP و ORDER BY ممنوعين فيه، وبيبطّأ الكتابة.
- الاختيار: B - View مع index على Publication(CitationCount DESC)، بسيط وداتا fresh.

Q52

- GRANT: يدي صلاحية.
- REVOKE: يشيل صلاحية اتدّت قبل كده ويرجع الوضع محايد.
- DENY: يمنع صراحةً وبياخد أولوية على أي GRANT.